

utiliser impérativement les feuilles quadrillées fournies pour les schémas

Une réponse non justifiée sera considérée comme fausse

Une image est composée d'un ensemble de pixels organisés sous forme d'un tableau. Pour une image en 'niveaux de gris', chaque pixel est codé sur un octet. La valeur 0 représente la couleur noire, la valeur 255 la couleur blanche. Les valeurs comprises entre 0 et 255 représentent les différentes nuances de gris. Une fonction de traitement d'images consiste à appliquer un certain algorithme à un tableau de pixels.

On souhaite étudier et comparer la performance d'exécution de la fonction **Seuil** sur deux réalisations de l'architecture Mips.

La première est la réalisation classique sur un pipeline de 5 étages présenté en cours. Cette réalisation est appelée **Mips**.

La seconde, appelée **SS2**, est la réalisation super-scalaire à 2 pipelines étudiée en cours.

La fonction **Seuil** vérifie si la valeur d'un pixel est supérieure à un certain seuil. Dans ce cas, le pixel est remplacé par un pixel blanc (code 255) et dans le cas contraire par un pixel noir.

```
void Seuil (unsigned char *img ,
            unsigned int   size ,
            unsigned char  seuil )
{
    for (i=0 ; i!=size ; i++)
    {
        if (img [i] >= seuil) img [i] = 255;
        else                  img [i] = 0;
    }
    return ;
}
```

On dispose de deux versions pour le code de la boucle principale de la fonction **Seuil** en assembleur Mips-32 pipeline. Conformément aux conventions utilisées par le compilateur Gcc, on suppose qu'à l'entrée de la boucle :

- le registre R4 pointe sur le tableau de pixels
- le registre R5 contient le nombre de pixels de l'image
- le registre R6 contient la valeur du seuil

De plus, le registre R8 contient l'adresse immédiatement après la fin du tableau de pixels (**img + size**) et le registre R7 la valeur 255.

Version 1 :

```
_For1 :      Lbu      r10, 0 (r4 )
             Sltu    r11, r10, r6
             Sb      r0 , 0 (r4 )
             Bne     r11, r0 , _Endif
             Nop
             Sb      r7 , 0 (r4 )
_Endif :     Addiu   r4 , r4 , 1
             Bne     r4 , r8 , _For1
             Nop
```

Version 2 :

```
_For2 :      Lbu      r10, 0 (r4 )  
             Sltu    r11, r10, r6  
             Addiu  r11, r11, -1  
             Sb      r11, 0 (r4 )  
             Addiu  r4 , r4 , 1  
             Bne    r4 , r8 , _For2  
             Nop
```

Dans tout l'exercice, on suppose qu'en moyenne 50% des pixels sont en dessous du seuil et que cette répartition est uniforme en tout point de l'image. L'image contient 4K pixels.

Dans un premier temps, on considère que la système mémoire est parfait (tous les accès mémoire s'effectuent en 1 cycle).

- 1- Pour la version 1, calculer le nombre moyen de cycles par itération sur la réalisation **Mips**. Il n'est pas demandé de faire un schéma, mais d'indiquer simplement sur le code les cycles de gel. Calculer le CPI et le CPI-utile.
- 2- Même question pour la version 2.
- 3- Pour la version 1, analyser l'exécution de la boucle à l'aide d'un schéma simplifié pour **SS2**. On suppose que l'étiquette **_For1** se trouve sur une adresse alignée (multiple de 8) et qu'au début de la boucle le buffer d'instructions est vide. Faire un schéma dans le cas où le branchement réussit et un schéma dans le cas où il échoue. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 4- Pour la version 2, analyser l'exécution de la boucle à l'aide d'un schéma simplifié pour **SS2**. On suppose que l'étiquette **_For1** se trouve sur une adresse alignée (multiple de 8) et qu'au début de la boucle le buffer d'instructions est vide. Calculer le nombre moyen de cycles par itération. Calculer le CPI et le CPI-utile.
- 5- Pour la version 1, optimiser le code de la boucle pour la réalisation **SS2** à l'aide de la technique **pipeline logiciel**.
- 6- Même question pour la version 2.

Dans le reste du sujet, on considère le code original des 2 versions pour la réalisation **Mips**.

On suppose maintenant que le processeur est relié à un cache de données et à un cache d'instructions séparés. Le cache d'instructions est parfait (taux de miss = 0).

Le cache de données a une capacité de 2Koctets. C'est un cache à correspondance directe (*Direct Mapping*) et un bloc du cache contient 16 octets.

Le cache est relié à la mémoire primaire à travers un bus. A chaque cycle, on peut transférer 4 octets (1 mots) sur le bus. Pour initialiser le transfert, il faut, en moyenne, 5 cycles. Pour charger un bloc manquant dans le cache, il faut donc 9 cycles ($5 + 1 \times \frac{16}{4}$)

- 7- Le cache de données contient combien d'emplacements ? Combien d'ensembles ?
Le tableau de pixels est constitué de combien de blocs ?

On souhaite évaluer la performance d'exécution des 2 versions de la fonction **Seuil** dans 2 hypothèses.

D'abord, on utilise un cache de données "*Write Through*" qui dispose d'un buffer d'écritures postées de 1 place. Le buffer est vidé à la fin de la transaction sur le bus.

On suppose qu'au début de l'exécution de la fonction le cache de données est vide.

- 8-** Pour la version 1, combien de Miss se produisent-ils lors du traitement de l'image ? Calculer le nombre de cycles supplémentaires par itération dus aux lectures.

Calculer le nombre cycles supplémentaires par itération dus aux écritures.

En déduire le nombre moyen de cycles par itération, le CPI et le CPI-utile.

- 9-** Même question pour la version 2.

On utilise un cache de données "*Write Back*". Pour une écriture, lorsque le bloc n'est pas présent dans le cache, le cache commence par charger le bloc manquant, puis le modifie.

- 10-** Pour la version 1, combien de Miss se produisent-ils lors du traitement de l'image à cause des lectures (distinguer 2 types de Miss) ? Calculer le nombre de cycles supplémentaires par itération dus au Miss.

Combien de Miss se produisent-ils lors du traitement de l'image à cause des écritures (distinguer 2 types de Miss) ? Calculer le nombre cycles supplémentaires par itération dus aux écritures.

En déduire le nombre moyen de cycles par itération, le CPI et le CPI-utile.

- 11-** Même question pour la version 2.